Product Requirements Document (PRD)

Product Requirements Document (PRD)
AI Philosophical Reflection Mentor
Inspired by Teachings of the Bhagavad Gita

---

1. Product Vision

The product is an AI-powered **philosophical reflection mentor** that helps users navigate life concerns through wisdom grounded in timeless teachings.

The initial philosophical foundation is derived from the **Bhagavad Gita**, one of the world's most influential spiritual texts.

The goal is not to create a chatbot that gives quick answers, but to build a **guided reflection experience** where users can think through their challenges with the help of philosophical insight.

The product acts as a **calm reflective mentor**, helping users gain clarity in moments of:

- anxiety
- confusion
- stress
- emotional conflict
- life decisions
- personal growth

Instead of providing generic advice, the mentor **guides the user through a philosophical reflection process inspired by the dialogue between Krishna and Arjuna.**

---

2. Product Philosophy

The mentor does not invent wisdom.

All guidance is grounded in **philosophical teachings and scripture interpretation**.

The mentor follows a structured reflection framework.

Reflection Structure

1. Understand the user's concern
2. Acknowledge the user's emotional state

3. Identify the philosophical conflict behind the concern
4. Introduce a relevant philosophical principle
5. Reference a teaching from the Bhagavad Gita
6. Interpret the teaching in modern language
7. Apply the teaching to the user's situation
8. Encourage reflection or action

The goal is to **guide thinking rather than simply comfort the user**.

Unlike many AI assistants that tend to validate user assumptions, the mentor may:

- challenge thinking
- reframe problems
- introduce new perspectives

This reflects how philosophical teaching traditionally works.

---

3. Product Principles

The product experience must feel:

- calm
- thoughtful
- reflective
- patient
- wise
- non-judgmental

The mentor must avoid:

- preaching
- lecturing
- robotic responses
- pretending to be Krishna

Instead, the mentor acts as a **philosophical interpreter and guide**.

---

4. Target Users

Primary Users

Young professionals who experience:

- stress
- career uncertainty
- overthinking
- decision paralysis

Secondary Users

College students exploring:

- purpose
- meaning
- identity
- direction in life

Tertiary Users

Individuals interested in:

- spirituality
- philosophy
- personal growth
- reflective thinking

---

5. Core Problem the Product Solves

Modern users often experience moments where they need **guidance, perspective, or emotional grounding**.

Typical moments include:

- late-night overthinking
- anxiety about career or relationships
- confusion about life direction
- feeling mentally overwhelmed

Most current tools provide:

- distraction
- quick advice

- surface-level motivation

The product instead provides **philosophical clarity through reflection.**

---

6. Core Product Experience

The primary interaction occurs inside a **mentor environment designed for calm reflection.**

The experience should not resemble a typical chat application.

Instead, it should feel like:

> sitting with a thoughtful mentor in a peaceful environment.

---

7. Mentor Environment

The mentor environment visually reinforces calm reflection.

Environment Design

The environment contains:

- a peaceful mountain or mound
- a monk-like mentor figure seated calmly
- flowing water in the background
- soft mist or natural movement
- subtle ambient animation

The visual design should evoke:

- peace
- stillness
- contemplation
- emotional safety

---

8. Mentor Representation

The mentor appears as a **neutral monk-like guide**.

Important rules:

- the mentor should NOT resemble Krishna
- the mentor represents wisdom, not a deity
- the mentor symbolizes philosophical guidance

This avoids religious misrepresentation while preserving symbolic meaning.

---

9. Interaction Modes

Users interact with the mentor through:

- voice conversation (primary)
- text input (optional)

Voice interaction helps create a **natural and emotionally comfortable experience**.

---

10. Conversation Flow

User interaction follows a simple flow.

User opens mentor environment → User shares concern → AI analyzes emotional and philosophical context → AI retrieves relevant teachings → Mentor responds with voice guidance → Text transcript appears gradually → Mentor may ask reflection questions → Conversation continues.

---

11. Voice Guidance

Mentor responses are delivered with:

- calm tone
- slow pacing
- thoughtful pauses
- conversational clarity

This supports moments such as:

- nighttime reflection
- emotional stress
- quiet contemplation.

---

12. On-Screen Transcription

While the mentor speaks, the response appears on screen as text.

Text should appear **gradually synchronized with voice**.

Benefits:

- improves accessibility
- allows reading alongside listening
- enables later reflection

---

13. Conversation Transcript

Each conversation maintains a transcript containing:

User message → Mentor response → User message → Mentor response

This allows the user to revisit earlier thoughts within the session.

---

14. Chat History

Users can revisit past conversations.

History includes:

- conversation summary
- date of conversation
- full transcript

Purpose:

- reflection on past insights
- continuity of thought

- long-term personal growth.

---

15. Knowledge System

The mentor's knowledge is grounded in **Bhagavad Gita teachings**.

The scripture is processed into:

- verse segments
- English meaning
- philosophical interpretation
- contextual explanations

When responding, the system retrieves teachings relevant to the user's situation.

The AI then explains the teaching in **modern language and practical context.**

This ensures responses remain grounded in authentic philosophy.

---

16. Philosophy-Guided Dialogue Model

The mentor conversation model follows the spirit of **Krishna-Arjuna dialogue structure**.

This includes:

Acknowledging confusion → questioning assumptions → introducing philosophy → clarifying duty or perspective → guiding toward clarity.

This structured approach differentiates the product from general chatbots.

---

17. Daily Reflection System

The product encourages daily philosophical reflection.

Users receive a **daily teaching or reflection prompt**.

Example prompts:

- What action today aligns with your duty?
- What expectation are you attached to today?
- Where can you act without fear of outcomes?

The goal is to make reflection a **daily habit**.

---

18. Habit Tracker

The application includes a **habit tracking system inspired by philosophical practice.**

Possible habit categories include:

- discipline
- reflection
- mindful action
- detachment from outcomes
- daily learning

Users can track progress and maintain consistency.

---

19. Personal Reflection Journal

Users can write personal reflections after conversations.

Journal entries may include:

- lessons learned
- personal insights
- questions for future reflection

The journal becomes a **record of philosophical growth.**

---

20. Shareable Insight Cards

After conversations, the system generates **shareable reflection cards**.

Cards may include:

- a relevant Gita teaching
- a simplified interpretation
- a reflection prompt

These cards can be shared on:

- WhatsApp
- Instagram
- social media platforms

This enables organic growth while spreading philosophical ideas.

---

21. Personalized Reflection Posters

Users can generate visual posters containing:

- philosophical quotes
- personal reflections
- calming visual themes

These can be downloaded or shared.

---

22. Language Support

The mentor supports multiple languages.

Primary languages:

- English
- Hindi

The system also tolerates regional dialects such as:

- Marwari
- Mewari
- Rajasthani variations

Speech may be interpreted internally as Hindi if needed.

---

23. Landing Website

The public website explains the product.

Sections include:

Hero section → Explanation of the idea → How the mentor works → Preview of the environment → Benefits of philosophical reflection → Call to action: Start Conversation.

Login is not required to explore the website.

---

24. Authentication

Users must authenticate before entering the mentor environment.

Supported login methods:

- Google login
- email login

Authentication allows:

- conversation history
- reflection storage
- habit tracking.

---

25. Feedback System

User feedback helps improve mentor quality.

Post-Conversation Feedback

After a session, users can rate the experience and provide comments.

Feedback Page

Users can submit:

- suggestions

- improvement ideas
- experience reports.

---

26. Safety and Ethical Guidelines

The mentor must handle sensitive emotional situations responsibly.

If a user expresses severe distress or self-harm thoughts:

- respond empathetically
- avoid philosophical lecturing
- encourage seeking professional support

The mentor must not claim to replace therapy or medical care.

---

27. Product Positioning

The product should be positioned as:

A calm AI reflection mentor inspired by timeless philosophical wisdom.

It is not:

- a religious preaching tool
- a chatbot impersonating Krishna
- a scripture reader

Instead it is:

a reflective space for thoughtful conversations grounded in philosophy.

---

28. Long-Term Product Vision

Over time, the mentor may expand beyond a single philosophical tradition.

Future philosophical foundations may include:

- Stoicism

- Vedanta
- Buddhism
- Taoism

The long-term goal is to create a **universal AI wisdom mentor for human reflection and growth.**

MVP

MVP — AI Philosophical Reflection Mentor
Inspired by Teachings of the Bhagavad Gita

---

1. MVP Objective

The goal of the MVP is to validate whether users find value in **philosophy-guided conversations for personal reflection**.

The MVP focuses on a single core question:

Will users talk to a philosophical mentor and feel calmer or more thoughtful afterward?

The MVP prioritizes:

- a calm mentor experience
- grounded responses based on Bhagavad Gita teachings
- voice and text conversations
- conversation history
- basic reflection sharing

The MVP intentionally avoids complex features so the core experience can be validated quickly.

---

2. Core Problem Being Tested

Many users experience moments where they need:

- emotional clarity
- philosophical perspective
- thoughtful guidance

Common scenarios include:

- late-night overthinking
- career confusion
- emotional stress
- relationship uncertainty
- feeling mentally overwhelmed

Current tools provide:

- generic advice
- motivational quotes
- distraction

The MVP tests whether users prefer **guided philosophical reflection instead.**

---

3. Core Value Proposition

The product offers:

A calm AI mentor that helps users think through life problems using teachings from the Bhagavad Gita.

The mentor:

- acknowledges emotions
- introduces philosophical insight
- explains teachings in modern language
- encourages reflection

This creates a **thinking process**, not just advice.

---

4. MVP Feature Scope

The MVP contains only the essential components needed to deliver the core experience.

Included features:

- landing page
- mentor environment
- conversation system
- Bhagavad Gita knowledge grounding
- voice responses
- text transcript
- conversation history
- shareable insight cards
- feedback system
- authentication

Excluded from MVP:

- habit tracker
- journaling system
- poster marketplace
- advanced analytics
- multi-philosophy knowledge base

These features are planned for future phases.

---

5. User Journey

Typical user flow:

User visits landing page → Reads about the product → Clicks "Start Conversation" → Signs in → Enters mentor environment → Shares a concern → Mentor responds with philosophical guidance → Conversation continues → User receives reflection insight → User may share insight or end session.

---

6. Landing Website

The landing website introduces the concept of the mentor.

Sections include:

Hero section → Explanation of philosophical reflection → How the mentor works → Visual preview of the mentor environment → Call-to-action: Start Conversation.

Purpose:

- communicate the idea
- build trust
- convert visitors into users.

---

7. Authentication

Users must log in before entering the mentor environment.

Supported login methods:

- Google login
- email login

Authentication enables:

- saving conversations
- retrieving chat history
- user feedback tracking.

---

8. Mentor Environment

The mentor environment is the **main product experience**.

The interface should feel calm and immersive.

Visual Elements

The environment includes:

- a peaceful mountain or mound landscape
- a monk-like mentor figure sitting calmly
- flowing water in the background
- soft natural motion

The goal is to create an atmosphere of:

- peace
- reflection
- emotional comfort.

---

9. Mentor Representation

The mentor appears as a **neutral monk-like guide**.

Important design rules:

- the mentor does not resemble Krishna
- the mentor represents wisdom, not a deity
- the mentor symbolizes thoughtful guidance.

---

10. Conversation Interface

Users can interact using:

- voice input
- text input

The interface displays a conversation transcript.

Conversation structure:

User message → Mentor response → User message → Mentor response.

The transcript helps maintain conversation context.

---

11. Conversation Flow

The mentor follows a structured reasoning process.

Conversation flow:

User shares concern → System analyzes emotional context → System retrieves relevant Bhagavad Gita teachings → Mentor explains philosophical idea → Mentor connects teaching to user's situation → Mentor may ask reflection question.

The response should feel **thoughtful and calm**, not robotic.

---

12. Voice Responses

Mentor responses are delivered using a calm voice.

Voice characteristics:

- slow pace
- thoughtful tone
- natural pauses
- clear articulation

Voice guidance enhances the reflective experience.

---

13. On-Screen Text Transcription

While the mentor speaks, the response text appears on screen.

The text should appear **gradually synchronized with voice playback**.

Benefits:

- accessibility
- easier understanding
- ability to revisit ideas.

---

14. Knowledge System

The mentor's knowledge is grounded in the **Bhagavad Gita**.

The scripture is processed into:

- verse chunks
- English translation
- contextual meaning
- philosophical explanation

The system retrieves relevant teachings during conversations.

The AI then interprets the teaching in modern language.

This prevents generic AI responses.

---

15. Conversation History

Users can revisit past conversations.

History includes:

- date
- conversation summary
- full transcript

Purpose:

- reflection on previous insights
- personal growth tracking.

---

16. Shareable Insight Cards

After a conversation, the system generates a **reflection insight card**.

The card contains:

- a Bhagavad Gita teaching
- a simplified interpretation
- a reflection prompt

Users can download or share the card on:

- WhatsApp
- Instagram
- other platforms.

This also supports organic product discovery.

---

17. Feedback System

User feedback is essential for improving the mentor.

Post-Conversation Feedback

After a session, users can provide:

- rating
- optional comment.

Feedback Page

Users can submit suggestions or report issues.

---

18. Language Support

The MVP supports two languages:

- English
- Hindi

The system should tolerate dialect variations such as:

- Marwari
- Mewari
- Rajasthani speech patterns.

Dialect speech may be interpreted internally as Hindi.

---

19. Safety Guidelines

The mentor must respond responsibly in sensitive situations.

If users express severe distress or self-harm thoughts:

- respond empathetically
- avoid philosophical lecturing
- encourage seeking professional support

The mentor must clearly avoid presenting itself as a replacement for therapy.

---

20. MVP Success Metrics

The MVP is considered successful if the following signals appear:

Users spend **10 minutes or more per session**

Users return for **multiple conversations**

Users provide **positive feedback about clarity or calmness**

Users share **insight cards externally**

---

21. Estimated Development Timeline

Suggested MVP development timeline:

Week 1 → Bhagavad Gita knowledge system and mentor response logic

Week 2 → conversation interface and voice responses

Week 3 → mentor environment UI and chat history

Week 4 → landing page, authentication, feedback system, testing

---

22. MVP Product Positioning

The MVP should be positioned as:

A calm AI reflection mentor inspired by teachings from the Bhagavad Gita.

It provides a space where users can think through life concerns using timeless philosophical insight.

The product is not:

- a religious preaching tool
- a scripture reader
- a chatbot impersonating Krishna.

Instead it is:

a reflective space for thoughtful conversations grounded in philosophical wisdom.

Tech Stack

Technical Architecture Document
MVP — AI Philosophical Reflection Mentor
Inspired by Teachings of the Bhagavad Gita

---

1. Objective

This document defines the **technical architecture and technology stack** for building the MVP of the AI Philosophical Reflection Mentor.

The primary technical goals are:

- Build a stable conversational product
- Ground responses using Bhagavad Gita teachings
- Enable voice interaction
- Maintain conversation history
- Ensure the entire system runs using **free and open-source technologies**

The architecture separates application layers to ensure maintainability and scalability.

---

2. System Architecture Overview

The system is divided into three major layers:

Frontend → Backend API → AI Service

The AI service handles all machine learning and reasoning tasks, while the backend manages application logic and data.

System architecture:

User → React Frontend → Spring Boot API → Python AI Service → RAG Pipeline → LLM (Ollama + LLaMA)

Supporting infrastructure:

- PostgreSQL database
- Vector database for scripture embeddings
- Local LLM inference via Ollama

---

3. Technology Stack

Frontend

Framework
React

Language
TypeScript

Purpose

- User interface
- Mentor environment rendering
- Voice input handling
- Displaying conversation transcripts
- Displaying insight cards
- Calling backend APIs

Libraries

React
TailwindCSS (UI styling)
Axios (API communication)

Browser APIs

Web Speech API (voice input)
Audio playback APIs (voice output)

Deployment

Vercel (free tier)

Backend API Layer

Framework
Java Spring Boot

Language
Java

Purpose

- API gateway between frontend and AI service
- Authentication management
- Chat session management
- Conversation history storage
- Feedback collection
- Business logic

Responsibilities

User authentication
Session management
Chat history persistence
Forwarding AI requests
Receiving AI responses

Deployment

Render / Railway (free tier)

AI Service

Framework
Python

Language
Python

Purpose

- Handle all AI processing
- Implement RAG pipeline
- Query Bhagavad Gita knowledge base
- Construct prompts
- Call LLM
- Return mentor responses

Frameworks and Libraries

FastAPI (API server)

LangChain or LlamaIndex (optional orchestration)
SentenceTransformers (embedding model)

Deployment

Render / Railway (free tier)

---

4. LLM Inference

Model runtime

Ollama

Purpose

Run LLM models locally for free without external API costs.

Model

LLaMA (open-source large language model)

Possible model versions

Llama 3 8B
Mistral 7B (alternative)

Ollama responsibilities

- local model inference
- prompt execution
- response generation

Advantages

No API cost
Full control over responses
Works offline
Easy local deployment

---

5. Knowledge System (RAG)

The AI mentor responses must be grounded in **Bhagavad Gita teachings**.

A Retrieval-Augmented Generation (RAG) pipeline will be used.

RAG pipeline:

User query → embedding generation → vector search → retrieve relevant verses → construct prompt → LLM response.

Scripture Data Preparation

Bhagavad Gita text will be processed into chunks containing:

Verse number
Sanskrit verse
English translation
Meaning / explanation

Each chunk will be embedded and stored in a vector database.

Example chunk structure:

Chapter
Verse
Translation
Meaning
Philosophical theme

---

6. Embedding Model

Embedding generation is required for semantic search.

Embedding library

SentenceTransformers

Embedding model

all-MiniLM-L6-v2

Advantages

Fast
Free
Open source
Good semantic similarity performance

---

7. Vector Database

The vector database stores embeddings for scripture chunks.

Technology

ChromaDB

Purpose

- store verse embeddings
- perform similarity search
- retrieve relevant teachings

Advantages

Open source
Runs locally
Easy Python integration
No infrastructure cost

---

8. Prompt Construction

The AI service constructs prompts using retrieved teachings.

Prompt structure:

System instruction
User message
Retrieved Bhagavad Gita verses
Philosophical interpretation context

Example reasoning structure enforced in prompts:

1. acknowledge the user's concern
2. introduce a philosophical perspective
3. reference a Bhagavad Gita teaching
4. explain it in modern language
5. encourage reflection

This ensures responses remain aligned with the product philosophy.

---

9. Conversation Flow

Conversation request lifecycle:

User message → React frontend sends request → Spring Boot backend receives message → Backend forwards request to Python AI service → AI service performs RAG retrieval → LLM generates response → AI response returned to backend → Backend sends response to frontend → Response displayed and spoken to user.

---

10. Voice Interaction

The product supports voice input and voice output.

Speech-to-Text

Speech-to-text will be handled using:

Web Speech API (browser)

Advantages

Free
Built into modern browsers
No server infrastructure required

Text-to-Speech

Voice output options:

Edge TTS
Coqui TTS

Purpose

Convert mentor responses into spoken guidance.

Desired voice characteristics

Calm
Slow paced
Clear articulation

---

11. Database

Application data will be stored in PostgreSQL.

Database technology

PostgreSQL

Recommended provider

Supabase (free tier)

Data stored includes:

User accounts
Conversation sessions
Chat transcripts
Feedback submissions

---

12. Conversation Storage

Each conversation session stores:

User ID
Session ID
Timestamp

User messages
Mentor responses

Purpose

- chat history
- reflection review
- product improvement analysis

---

13. Shareable Insight Cards

After conversations, the system generates reflection insight cards.

Card generation process:

Conversation summary → identify key teaching → create shareable card → allow download or sharing.

Card generation may be implemented via:

HTML template rendering
Canvas image generation

---

14. Deployment Strategy

The system will be deployed using free-tier services.

Frontend

Vercel

Backend

Render / Railway

AI Service

Render / Railway

Database

Supabase (PostgreSQL)

Vector Database

ChromaDB (local or containerized)

LLM

Ollama running on AI service host

---

15. Development Environment

Local development setup:

Frontend
Node.js environment

Backend
Java + Maven

AI Service
Python + virtual environment

LLM
Ollama running locally

Database
Local PostgreSQL or Supabase connection

---

16. Security Considerations

Authentication

Google OAuth
Email authentication

API communication

HTTPS only

Sensitive data

User conversations should not be exposed publicly.

---

17. Monitoring

Basic monitoring includes:

API logs
AI service logs
conversation error tracking

Tools

Spring Boot logs
Python logs
Render platform logs

---

18. Future Technical Extensions

Possible future improvements:

- streaming AI responses
- multi-philosophy knowledge bases
- improved voice synthesis
- habit tracking engine
- journaling system
- advanced RAG ranking

These extensions are not required for the initial MVP.

---

19. Summary

The MVP technical architecture uses a **three-layer system**:

React frontend
Spring Boot backend

Python AI service

AI capabilities are powered by:

RAG pipeline
ChromaDB vector search
SentenceTransformer embeddings
Ollama running LLaMA locally

The entire system is designed to operate using **free and open-source tools** while providing a scalable architecture for future expansion.

Tech Task

MVP Development Task Breakdown
AI Philosophical Reflection Mentor

This document breaks the MVP into **feature-based vertical slices**.
Each feature includes **Frontend, Backend, and AI tasks** so the system can be built and integrated incrementally.

Development order should follow the feature sequence to ensure working end-to-end functionality early.

Feature 1 — Project Setup & Base Infrastructure

Goal: Establish repositories, development environments, and core project structure.

Frontend Tasks

- Initialize React project using Vite
- Setup TypeScript configuration
- Install core dependencies
  - axios
  - react-router
  - tailwindcss
- Create base folder structure

Example structure:

```
src/
components/
pages/
services/
hooks/
utils/
```

- Configure environment variables
- Setup global API client
- Setup basic layout component

Backend Tasks

- Initialize Spring Boot project

- Setup Maven dependencies
  - Spring Web
  - Spring Security
  - Spring Data JPA
  - PostgreSQL driver
- Configure application.yml
- Setup database connection
- Create basic project structure

Example structure:

controller/
service/
repository/
entity/
config/

- Enable CORS for frontend communication

AI Tasks

- Create Python AI service
- Setup FastAPI server
- Define project structure

Example:

ai_service/
main.py
rag/
embeddings/
vector_store/
llm/
prompts/

- Setup virtual environment
- Install dependencies
  - fastapi
  - sentence-transformers
  - chromadb
  - ollama client

Feature 2 — Authentication

Goal: Allow users to log in and maintain session identity.

Frontend Tasks

- Create login page
- Implement Google OAuth login
- Implement email login form
- Store authentication token
- Handle redirect after login
- Protect private routes

Routes

/login
/chat
/history

Backend Tasks

- Implement authentication controller

POST /auth/google
POST /auth/login

- Implement JWT generation
- Create User entity

User fields

id
email
name
created_at

- Implement authentication middleware
- Secure protected APIs

AI Tasks

None required for this feature.

Feature 3 — Chat Session Initialization

Goal: Allow users to start a new mentor conversation session.

Frontend Tasks

- Create mentor environment page
- Create "Start Conversation" button
- Initialize chat session when page loads
- Display empty conversation UI

Backend Tasks

Create API

POST /chat/session

Response

```
{
  "sessionId": "uuid"
}
```

- Create ChatSession entity

Fields

session_id
user_id
created_at

- Persist session to database

AI Tasks

Optional task:

Create initial mentor greeting prompt.

Example output

"Welcome. What has been on your mind lately?"

Expose endpoint

POST /ai/greeting

Feature 4 — Chat Messaging

Goal: Enable user to send messages and receive mentor responses.

Frontend Tasks

- Build chat input component
- Display user message bubbles
- Display mentor response bubbles
- Show loading indicator
- Send messages to backend

API call

POST /chat/message

Backend Tasks

Create API

POST /chat/message

Request

```
{
  "sessionId": "...",
```

```
  "message": "..."
}
```

Flow

Receive message
Forward to AI service
Return AI response

Store message in database.

Create Message entity

Fields

id
session_id
role (user / mentor)
message
timestamp

AI Tasks

Create endpoint

POST /ai/chat

Request

```
{
  "message": "..."
}
```

Pipeline

User message
Embedding generation
Vector search
Retrieve Gita verses
Prompt construction
LLM response generation

Return response

```
{
  "response": "mentor response text"
}
```

Feature 5 — RAG Knowledge System

Goal: Ground responses in Bhagavad Gita teachings.

AI Tasks

Prepare dataset

Bhagavad Gita verses
English translation
Meaning

Chunk data

Example metadata

chapter
verse
translation
commentary

Generate embeddings

Store in ChromaDB.

Implement retrieval function

Input

User query

Output

Top 5 relevant verses.

Backend Tasks

No additional tasks required.

Frontend Tasks

No additional tasks required.

Feature 6 — Conversation History

Goal: Allow users to revisit past conversations.

Frontend Tasks

Create history page.

Display:

conversation list
date
summary

Allow clicking conversation to open transcript.

Backend Tasks

Create APIs

GET /chat/history

GET /chat/history/{sessionId}

Return conversation sessions and messages.

AI Tasks

Optional

Generate conversation summary using LLM.

Feature 7 — Insight Cards

Goal: Generate shareable reflection insights.

Frontend Tasks

Create insight card UI.

Display

teaching quote
reflection message

Allow

download
share.

Backend Tasks

Create API

POST /chat/insight

Request

sessionId

Forward conversation to AI service.

AI Tasks

Generate insight card content.

Steps

Analyze conversation
Extract philosophical theme
Generate short reflection.

Return

```
{
  "quote": "...",
  "reflection": "..."
}
```

Feature 8 — Voice Interaction

Goal: Allow users to speak to the mentor and hear responses.

Frontend Tasks

Implement voice input using Web Speech API.

Steps

Start microphone
Capture speech
Convert to text
Send to chat API.

Implement text-to-speech playback.

Backend Tasks

No additional tasks required.

AI Tasks

Ensure responses are optimized for spoken delivery.

Example

Short sentences
Natural pacing.

Feature 9 — Feedback System

Goal: Collect user feedback.

Frontend Tasks

Create feedback modal after conversation.

Fields

rating
comment

Backend Tasks

Create API

POST /feedback

Create Feedback entity

Fields

id
user_id
rating
comment
created_at

AI Tasks

None required.

Feature 10 — Deployment

Goal: Deploy MVP to accessible environment.

Frontend

Deploy on Vercel.

Backend

Deploy Spring Boot on Render.

AI Service

Deploy Python AI service on Render.

Infrastructure

Database

Supabase PostgreSQL

Vector Database

ChromaDB

LLM Runtime

Ollama running with LLaMA model.

Development Strategy

Each feature must be implemented **vertically**.

Process

Frontend task
Backend API
AI service implementation
Integration test

Move to next feature only after integration succeeds.

This approach prevents late-stage integration failures and ensures the product remains functional throughout development.

Design Doc

ClarityMantra — UI Design System & Theme Guide

1. Purpose

This document defines the **visual identity, theme, and UI principles** for the ClarityMantra product.

It ensures that all future screens maintain **consistent design, colors, typography, spacing, and product feeling**.

The product should visually communicate:

- Calm
- Clarity
- Reflection
- Wisdom
- Simplicity

The UI should feel closer to **Calm / Headspace / Apple Health** rather than a typical chatbot interface.

---

2. Design Philosophy

ClarityMantra is **not a chatbot UI**.

The product experience should feel like:

> A calm space to think.

Design principles:

- Minimal interface
- Calm visual tone
- Generous whitespace
- Soft rounded UI elements
- Nature-inspired colors
- No aggressive colors

- No clutter

The UI should feel **slow, thoughtful, and peaceful**.

---

3. Color Palette

Primary colors should reflect **calm and clarity**.

Primary Color

Deep Indigo
HEX: `#1E2A38`

Used for:

- Primary buttons
- Headings
- Navigation emphasis

Secondary Color

Sage Green
HEX: `#6C8A73`

Used for:

- accents
- icons
- calm highlights

Background Colors

Soft Sand
HEX: `#F3EFE8`

Light Beige
HEX: `#F8F6F2`

Used for:

- page backgrounds
- card surfaces

Accent Color

Muted Gold
HEX: `#C8A46C`

Used for:

- logo highlights
- subtle separators
- special quotes

Neutral Text

Dark Charcoal
HEX: `#333333`

Secondary Text
HEX: `#6B6B6B`

---

4. Typography

Typography should feel **modern, calm, and readable**.

Primary Font

Inter

Used for:

- body text
- navigation
- UI elements

Heading Font

Inter SemiBold / Bold

Used for:

- hero text
- section titles
- key headings

Sanskrit / Hindi Quotes

Font:

Noto Sans Devanagari

Used only for:

- philosophical quotes
- occasional wisdom text

Never use Devanagari for navigation or buttons.

---

5. Layout System

Use a **centered content layout**.

Page width:

Max width: 1200px

Spacing:

Section padding: 80px vertical
Component spacing: 24px

Card spacing:

16px – 24px

---

6. Button Style

Buttons should feel **calm and inviting**.

Primary Button:

Background: Deep Indigo
Text: White
Border Radius: 12px
Padding: `14px 28px`

Example label:

Start Conversation

Buttons should include **slight hover elevation**.

---

7. Icon Style

Icons should be:

- minimal
- line-based
- rounded

Recommended icon sets:

- Lucide
- Heroicons
- Feather Icons

Avoid heavy or filled icons.

---

8. Imagery Style

Images should feel:

- calm
- nature-inspired
- soft gradients
- watercolor / minimal illustration

Allowed imagery:

- mountains
- water
- sunrise
- meditation silhouette
- natural landscapes

Avoid:

- religious imagery
- temple imagery
- deity imagery

---

9. Sanskrit / Hindi Quote Component

The landing page and some screens may show a rotating philosophical quote.

Structure:

Sanskrit Quote
English Meaning
Source

Example:

कर्मण्येवाधिकारस्ते मा फलेषु कदाचन।

You have the right to perform your duties,
but not to the fruits of your actions.

— Bhagavad Gita

This component should appear:

- on landing page
- on reflection cards
- occasionally in mentor responses

---

10. Core Components

All screens should use the same UI components.

Core components:

- Navbar
- Hero section
- Conversation UI
- Insight card
- Quote card
- Feature cards
- CTA section
- Footer

These components should **reuse the same spacing and styles**.

---

11. Navigation Bar

Navbar contains:

Left:

Logo + ClarityMantra text

Right:

- How It Works
- About
- Start Conversation (Primary Button)

Navbar should be **minimal and fixed**.

---

12. Hero Section

Hero section structure:

Left side:

- Headline
- Subheadline
- CTA button

Right side:

- product preview
- conversation mockup

Example headline:

Find Clarity in Every Conversation

---

13. Feature Cards

Feature cards should be simple and calm.

Card structure:

Icon
Title
Short description

Example:

Reflect Deeply
Express what’s on your mind without fear or judgment.

---

14. Mentor UI Style

Conversation interface should feel:

- clean
- calm
- spacious

Avoid typical chatbot UI like:

- left sidebar
- conversation lists

Instead show:

mentor environment + conversation flow.

---

15. CTA Philosophy

CTA should be **very simple**.

Primary CTA:

Start Conversation

Users should be able to start using the product in **one click**.

---

16. Future Screen Consistency

All future screens must follow:

- same color palette
- same typography
- same button styles

- same spacing system

When designing new screens, this document should be used as the **reference guide**.

---

17. Brand Feeling Summary

ClarityMantra should visually feel like:

Calm
Wise
Minimal
Indian-rooted but modern
Technology + Philosophy

Not like:

Chatbot
Therapy app
Religious website
Meditation blog

---

18. Design Reference Products

The visual tone should be closest to:

- Calm
- Headspace
- Apple Health
- Notion (clean UI)

---

19. Usage

When generating or designing new screens in future:

1. Share this design document
2. Share the previous screen
3. Ask for a new screen

This ensures **consistent UI generation across threads**.

ClarityMantra

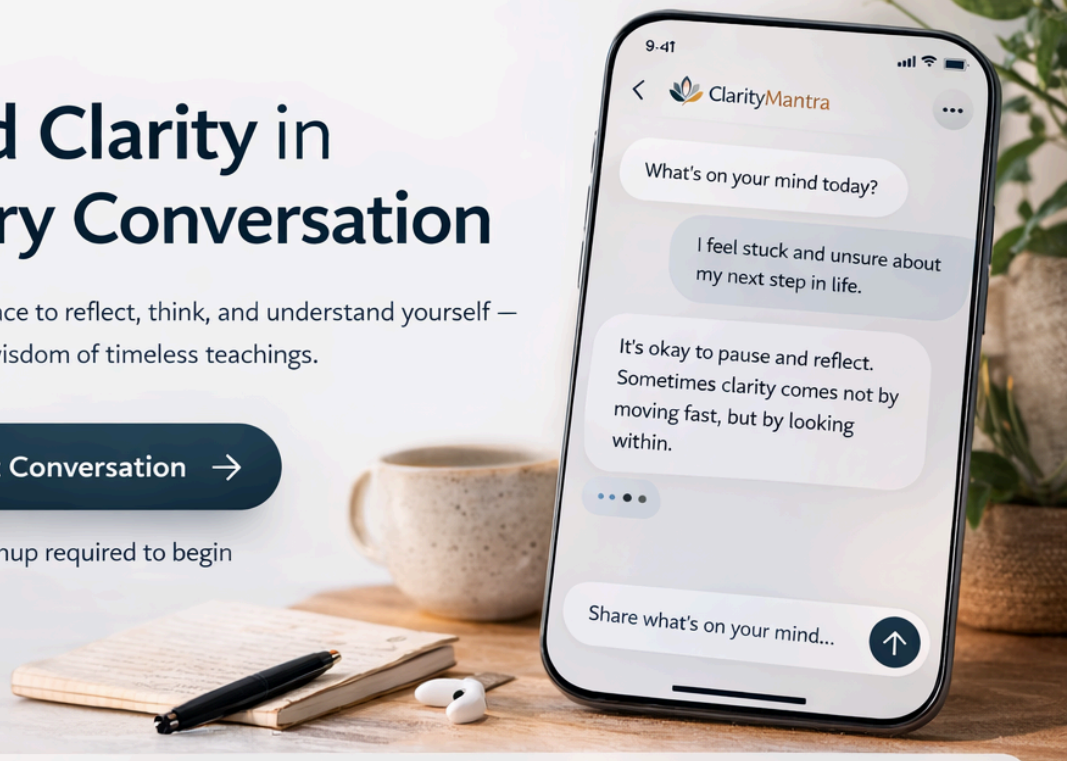
How It Works | About | Insights | Start Conversation

Find Clarity in Every Conversation

A calm space to reflect, think, and understand yourself — with the wisdom of timeless teachings.

Start Conversation →

No signup required to begin



कर्मण्येवाधिकारस्ते मा फलेषु कदाचन ।

You have the right to perform your duties, but not to the fruits of your actions.

— Bhagavad Gita

How It Helps

Reflect Deeply

Express what's on your mind without fear or judgment

Gain Perspective

See your thoughts with clarity and wisdom

Find Answers Within

Explore what truly matters to you

Feel Calm & Grounded

One step at a time

Start your journey to clarity

A conversation can change the way you see life.

Start Conversation →

Mind • Wisdom • Growth • Peace

Privacy | Terms | Contact

Backend APIs

ClarityMantra — Backend API Specification (MVP)

Architecture Summary

Frontend communicates with the backend using **REST APIs**.
The AI response is streamed using **Server-Sent Events (SSE)**.

Speech recognition uses **Whisper via audio upload (HTTP POST)**.

System layers:

```
Frontend (React)
      │
      │ REST / SSE
      ▼
Spring Boot Backend
      │
      │ HTTP
      ▼
Python AI Service
      │
      ▼
LLM + Whisper + RAG
```

---

1. Authentication APIs

1.1 Google Login

POST /auth/google

Request

```
{
  "idToken": "google_id_token"
}
```

Response

```
{
  "userId": "uuid",
  "email": "user@email.com",
  "name": "User Name",
  "jwt": "jwt_token"
}
```

Backend Actions

1. Verify token with Google
2. Create user if not exists
3. Issue JWT

---

2. Chat Session APIs

2.1 Create Chat Session

POST /chat/session

Headers

Authorization: Bearer JWT

Response

```
{
  "sessionId": "uuid",
  "createdAt": "timestamp"
}
```

Backend Actions

- Insert record into `chat_session` table.

---

3. Chat Messaging APIs

3.1 Send Message

POST /chat/message

Request

```
{
  "sessionId": "uuid",
  "message": "I feel stuck in my career"
}
```

Backend Actions

1. Store user message
2. Retrieve last N messages for context
3. Call AI service
4. Store AI response

Response

```
{
  "status": "accepted"
}
```

---

4. AI Response Streaming (SSE)

4.1 Stream AI Response

GET /chat/stream/{sessionId}

Headers

Accept: text/event-stream

Response Stream

data: It
data: sounds
data: like
data: you
data: are
data: feeling
data: uncertain
data: [DONE]

Backend Actions

1. Build prompt from conversation context
2. Call LLM
3. Stream tokens to client

Connection closes after `[DONE]`.

---

5. Conversation History APIs

5.1 Get Conversation List

GET /chat/history

Response

```
[
  {
    "sessionId": "uuid",
    "createdAt": "2026-03-15",
    "summary": "Career uncertainty reflection"
  },
  {
    "sessionId": "uuid",
    "createdAt": "2026-03-10",
    "summary": "Relationship reflection"
  }
```

```
]
```

5.2 Get Conversation Transcript

GET /chat/history/{sessionId}

Response

```
{
  "sessionId": "uuid",
  "messages": [
    {
      "role": "user",
      "message": "I feel stuck in my career",
      "timestamp": "..."
    },
    {
      "role": "mentor",
      "message": "It sounds like you are feeling uncertain...",
      "timestamp": "..."
    }
  ]
}
```

---

6. Insight Generation API

6.1 Generate Reflection Insight

POST /chat/insight

Request

```
{
  "sessionId": "uuid"
}
```

Response

```
{
  "quote": "कर्मण्येवाधिकारस्ते मा फलेषु कदाचन",
  "meaning": "You have the right to perform your duties but not the fruits of your actions.",
  "reflection": "Focus on the action you can take today rather than worrying about outcomes."
}
```

---

7. Speech Recognition (Whisper)

7.1 Audio Transcription

POST /speech/transcribe

Request

Content-Type: multipart/form-data

Payload:

audio_file

Response

```
{
  "text": "I feel confused about my career path"
}
```

Backend Actions

1. Send audio to Whisper
2. Return transcript text

---

8. Feedback API

8.1 Submit Feedback

POST /feedback

Request

```
{
  "sessionId": "uuid",
  "rating": 5,
  "comment": "Helpful reflection session"
}
```

Response

```
{
  "status": "saved"
}
```

---

9. Optional User API

9.1 Get Current User

GET /user/me

Response

```
{
  "userId": "uuid",
  "name": "User",
  "email": "user@email.com"
}
```

Database Tables

user

id
email
name
created_at

chat_session

session_id
user_id
created_at

message

id
session_id
role
message
created_at

feedback

id
session_id
rating
comment
created_at

Recommended index:

(session_id, created_at)

API Summary

Category	Endpoint
Auth	POST /auth/google

Session	POST /chat/session
Chat	POST /chat/message
Streaming	GET /chat/stream/{sessionId}
History	GET /chat/history
Transcript	GET /chat/history/{sessionId}
Insight	POST /chat/insight
Speech	POST /speech/transcribe
Feedback	POST /feedback
User	GET /user/me

Total APIs: **10**

AI Service

ClarityMantra — Python AI Service API Specification (MVP)

This document defines the **Python AI Service** responsible for all AI-related processing in the ClarityMantra system.

The AI service performs:

- LLM inference
- Retrieval Augmented Generation (RAG)
- Bhagavad Gita verse retrieval
- Insight generation
- Speech transcription (Whisper)

The service is **not exposed to the public internet**.
Only the **Spring Boot backend** calls these APIs.

---

1. Service Architecture

```
Spring Boot Backend
      │
      │ HTTP API
      ▼
Python AI Service (FastAPI)
      │
      ├─ Whisper (Speech to Text)
      ├─ RAG Retrieval
      ├─ Prompt Construction
      └─ LLM (Ollama / Llama)
```

Primary responsibilities:

speech transcription
chat reasoning
philosophical reflection generation
insight extraction

---

2. Technology Stack

Component	Technology
API Framework	FastAPI
LLM Runtime	Ollama
Embedding Model	sentence-transformers
Vector Database	ChromaDB
Speech Recognition	Whisper
Language	Python

---

3. Base URL

http://ai-service:8000

---

4. Health Check API

Used by backend services and deployment environments.

Endpoint

GET /health

Response

```
{
  "status": "ok",
  "service": "ai-service"
}
```

---

5. Chat AI API

This endpoint generates the mentor's response using:

- conversation context
- Bhagavad Gita retrieval
- structured philosophical reasoning

Endpoint

POST /ai/chat

Request

```
{
  "message": "I feel stuck in my career",
  "context": [
    {
      "role": "user",
      "message": "I feel confused about my future"
    },
    {
      "role": "mentor",
      "message": "What part of the future feels uncertain?"
    }
  ]
}
```

Processing Steps

1. Embed user query
2. Search vector database for relevant Gita verses
3. Build structured prompt
4. Send prompt to LLM
5. Return mentor response

Response

```
{
```

"response": "It sounds like you are feeling uncertain about the direction ahead. In the Bhagavad Gita, there is a teaching that reminds us that clarity often comes when we focus on our actions rather than outcomes."
}

---

6. Streaming Chat API (Optional)

If token streaming is implemented.

Endpoint

POST /ai/chat/stream

Request

{
"message": "I feel overwhelmed by my responsibilities",
"context": []
}

Response

Stream tokens:

data: It
data: sounds
data: like
data: you
data: are
data: feeling
data: overwhelmed
data: [DONE]

Content-Type:

text/event-stream

---

7. Insight Generation API

This endpoint extracts a philosophical teaching from a conversation.

Endpoint

POST /ai/insight

Request

```
{
  "conversation": [
    {
      "role": "user",
      "message": "I feel stuck in my career"
    },
    {
      "role": "mentor",
      "message": "It sounds like uncertainty about the future is weighing on you."
    }
  ]
}
```

Processing Steps

1. Analyze conversation theme
2. Retrieve relevant Gita verse
3. Generate reflection

Response

```
{
  "quote": "कर्मण्येवाधिकारस्ते मा फलेषु कदाचन",
  "meaning": "You have the right to perform your duties but not the fruits of your actions.",
  "reflection": "Focus on the actions you can take today rather than worrying about the
outcomes."
}
```

---

8. Speech Transcription API (Whisper)

Used for converting recorded speech to text.

Endpoint

POST /speech/transcribe

Request

multipart/form-data

Field:

audio_file

Example payload:

audio_file: user_recording.wav

Processing Steps

1. Receive audio file
2. Pass audio to Whisper
3. Return transcript

Response

```
{
  "text": "I feel uncertain about my career path"
}
```

---

9. Verse Retrieval API (Optional)

Returns relevant Bhagavad Gita verses.

Endpoint

POST /ai/retrieve

Request

```
{
  "query": "career confusion"
}
```

Processing Steps

1. Generate embedding
2. Vector search in ChromaDB
3. Return top verses

Response

```
{
  "results": [
    {
      "chapter": 2,
      "verse": 47,
      "translation": "You have the right to perform your duties, but not to the fruits of your actions."
    }
  ]
}
```

---

10. Vector Indexing API (Admin Only)

Used during dataset preparation.

Endpoint

POST /ai/index

Request

```
{
  "chapter": 2,
  "verse": 47,
  "translation": "You have the right to perform your duties...",
  "meaning": "Act without attachment to results."
}
```

Response

```
{
  "status": "indexed"
}
```

---

11. Prompt Structure Used by AI

All prompts follow this reasoning framework:

1. Acknowledge user concern
2. Identify emotional state
3. Introduce philosophical perspective
4. Reference Bhagavad Gita teaching
5. Explain teaching in modern context
6. Encourage reflection

---

12. AI Service Folder Structure

```
ai_service/
│
├── main.py
│
├── api/
│   ├── chat.py
│   ├── insight.py
│   ├── speech.py
│   └── health.py
│
├── rag/
│   ├── retriever.py
│   ├── embeddings.py
│   └── vector_store.py
│
├── llm/
│   ├── ollama_client.py
│   └── prompt_builder.py
│
```

```
├── models/
│   └── request_models.py
│
└── utils/
    └── logger.py
```

---

13. Expected Performance

Typical request lifecycle:

```
API request received
↓
vector search (10–30 ms)
↓
prompt construction
↓
LLM inference (2–5 seconds)
↓
response returned
```

Most latency comes from **LLM inference**.

---

14. Security

AI service should only be accessible by the backend network.

Recommended configuration:

```
private network
internal service discovery
```

Never expose this service directly to the internet.

---

15. Summary

The Python AI service exposes the following APIs:

API	Purpose
GET /health	service status
POST /ai/chat	generate mentor response
POST /ai/chat/stream	streaming AI response
POST /ai/insight	generate reflection insight
POST /speech/transcribe	Whisper transcription
POST /ai/retrieve	verse retrieval
POST /ai/index	vector indexing

These APIs form the **entire AI layer** of the ClarityMantra MVP.